DATA ANALYSIS · EVERY RACE, 1950–2024

WHAT WINS FORMULA 1 RACES?

01

The driver, the car, or the pit crew?

SQL · window functions — f1_project.ipynb

```
In [ ]: dominant = sql("""
-- top team per decade
WITH ranked AS (
    SELECT *,
           SUM(wins) OVER (PARTITION BY decade) AS races_in_decade,
           RANK() OVER (PARTITION BY decade
                        ORDER BY wins DESC)     AS rnk
    FROM team_wins
)
SELECT decade, team,
       ROUND(100.0 * wins / races_in_decade, 1) AS share
FROM ranked
WHERE rnk = 1
""")
```

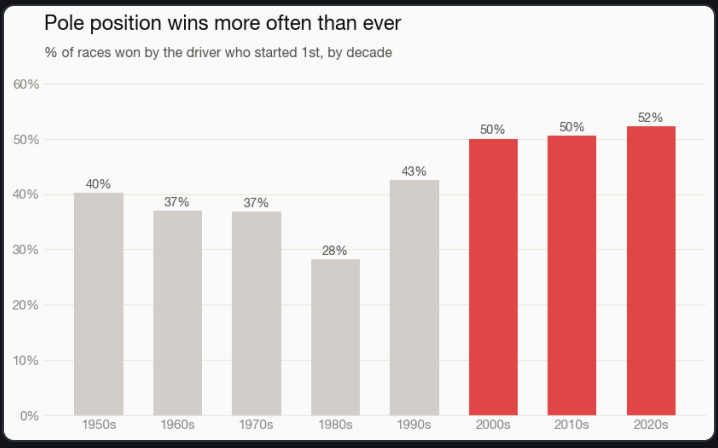
f1.db — 14 tables

○ lap_times	589,081
○ driver_standings	34,863
● results	26,759
○ constructor_standings	13,391
● pit_stops	11,371
● races	1,125
● drivers	861
● constructors	212

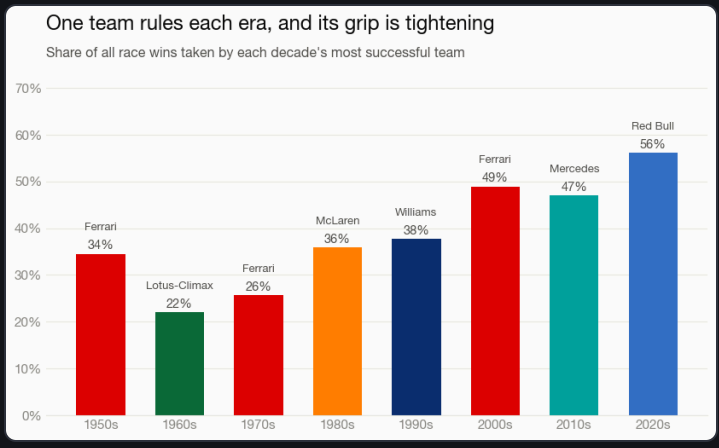
• used in this analysis · +6 more

total rows 701,433

Out[ ]: pole position by decade



Out[ ]: top team per decade



14 CSV FILES → 701,433 ROWS → 4 INSIGHTS

🏎️ What Wins Formula 1 Races?

The driver, the car, or the pit crew? Turning 14 raw CSV files into insights with SQL and Python

Author: Alex Santos · **Tools:** Python (pandas, matplotlib, SciPy), SQL (SQLite)

Data: Formula 1 World Championship dataset, every race since 1950

Key findings

1. **Pole position wins about half of all races since 2000**, up from 28–40% in earlier decades.
2. **The car does most of the work:** teammates finish 3.3 places apart vs. 6.3 for rivals.
3. **Faster pit stops go with better finishes:** 6.9th vs. 12.5th on average.
4. **Dominance is rising:** the top team now wins roughly half of all races.

1. From raw data to a database

The process: `14 CSV files` → **Python** loads them and fixes missing values → **SQLite database** → **SQL** joins, filters, and aggregates → **Python** analyzes and charts → **insights**

The raw data came as 14 separate CSV files (races, drivers, teams, results, pit stops, and more). On their own they can't be combined, so I loaded them into a single SQLite database where SQL can join them through their shared ID columns.

Setup

Libraries, folder paths, and one shared chart style so every chart looks consistent.

In [110…
```
import sqlite3, logging
from pathlib import Path
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.ticker as mtick
from scipy.stats import spearmanr
```

In [112…
```
DATA_DIR, DB_PATH, IMG_DIR = Path("f-1_data"), Path("f1.db"), Path("images")
IMG_DIR.mkdir(exist_ok=True)
logging.getLogger("matplotlib.font_manager").setLevel(logging.ERROR)
```

In [116…
```
SURFACE, INK, INK_2 = "#fcfcfb", "#0b0b0b", "#52514e"
GRID, NEUTRAL = "#e7e6e2", "#d3d1ca"
RED, BLUE = "#e34948", "#2a78d6"
BLUE_RAMP = ["#1c5cab", "#2a78d6", "#5598e7", "#86b6ef"]
TEAM_COLORS = {
    "Ferrari": "#DC0000", "Mercedes": "#00A19C", "Red Bull": "#3671C6",
    "McLaren": "#FF8000", "Williams": "#0B2D72", "Lotus": "#0E6B3A",
    "Brabham": "#5E8C31", "Cooper": "#1F5F3F", "Tyrrell": "#1B3C8C",
}

def team_color(name):
```

```
    for key, color in TEAM_COLORS.items():
        if name.replace("Team ", "").startswith(key):
            return color
    return "#9a9893"
```

In [120…

```
plt.rcParams.update({
    "font.family": ["Arial", "DejaVu Sans"],
    "font.size": 11,
    "figure.facecolor": SURFACE,
    "axes.facecolor": SURFACE,
    "savefig.facecolor": SURFACE,
    "axes.edgecolor": GRID,
    "axes.labelcolor": INK_2,
    "axes.spines.top": False,
    "axes.spines.right": False,
    "axes.spines.left": False,
    "axes.grid": True,
    "axes.grid.axis": "y",
    "axes.axisbelow": True,
    "grid.color": GRID,
    "xtick.color": "#8a8984",
    "ytick.color": "#8a8984",
    "xtick.major.size": 0,
    "ytick.major.size": 0,
})

def new_chart(title, subtitle, figsize=(10, 5.2)):
    fig, ax = plt.subplots(figsize=figsize)
    ax.text(0, 1.13, title, transform=ax.transAxes,
            fontsize=17, fontweight="bold", color=INK)
    ax.text(0, 1.05, subtitle, transform=ax.transAxes,
            fontsize=11.5, color=INK_2)
    return fig, ax

def label_bars(ax, bars, fmt):
    for bar in bars:
        x = bar.get_x() + bar.get_width() / 2
        y = bar.get_height()
        ax.annotate(fmt.format(y), (x, y), xytext=(0, 4),
                    textcoords="offset points", ha="center",
                    fontsize=10.5, fontweight="bold", color=INK_2)

def save(fig, name):
    fig.savefig(IMG_DIR / f"{name}.png", dpi=200, bbox_inches="tight")
```

In [124…

```
conn = sqlite3.connect(DB_PATH)
loaded = []

for csv in sorted(DATA_DIR.glob("*.csv")):
    df = pd.read_csv(csv, na_values=["\\N"])
    df.to_sql(csv.stem, conn, if_exists="replace", index=False)
    loaded.append({"table": csv.stem, "rows": len(df),
                   "columns": len(df.columns)})

def sql(query):
    return pd.read_sql(query, conn)

summary = pd.DataFrame(loaded)
total_rows = summary["rows"].sum()
print(f"Loaded {len(summary)} CSV files → {total_rows:,} rows in one database")
summary.style.format({"rows": "{:,}"}).hide(axis="index")
```

```
Loaded 14 CSV files → 701,433 rows in one database
```

Out[124…

table	rows	columns
circuits	77	9
constructor_results	12,625	5
constructor_standings	13,391	7
constructors	212	5
driver_standings	34,863	7
drivers	861	9
lap_times	589,081	6
pit_stops	11,371	7
qualifying	10,494	9
races	1,125	18
results	26,759	18
seasons	75	2
sprint_results	360	16
status	139	2

In [128…

```
results = sql("SELECT * FROM results")

print("Rows:", f"{len(results):,}")
print("Duplicate rows:", results.duplicated().sum())
print("Duplicate resultIds:", results["resultId"].duplicated().sum())
print("\nMissing values per column (%):")
(results.isna().mean() * 100).round(1).sort_values(ascending=False).head(8)
```

```
Rows: 26,759
Duplicate rows: 0
Duplicate resultIds: 0

Missing values per column (%):
```

Out[128…

```
milliseconds       71.3
time               71.3
fastestLapSpeed    69.2
fastestLapTime     69.2
fastestLap         69.2
rank               68.2
position           40.9
resultId            0.0
dtype: float64
```

Data quality: No duplicate rows or result IDs. The gaps are expected: race time and fastest-lap data only exist for finishers and more recent seasons, and `position` is blank (41%) for drivers who didn't finish. The analysis filters on `position IS NOT NULL` wherever only finishers should count.

How the tables connect: `results` is the center of the dataset, one row per driver per race. It links to the rest through shared IDs:

Table	Joined on	Used for
`races`	`raceId`	season and race name
`drivers`	`driverId`	driver names
`constructors`	`constructorId`	team names
`pit_stops`	`raceId` + `driverId`	pit stop times (2011+)

SQL techniques used: multi-table joins, a self-join, `CASE WHEN`, CTEs (`WITH`), and window functions (`RANK() OVER`, `SUM() OVER`).

2. Is it the car or the driver?

Two tests: how often the fastest qualifier wins, and how close teammates in identical cars finish.

In [184…
```
winners = sql("""
SELECT ra.year, r.grid AS start_position
FROM results r
JOIN races ra ON ra.raceId = r.raceId
WHERE r.positionOrder = 1
ORDER BY ra.year
""")

winners.head()
```

Out[184…

	year	start_position
0	1950	1
1	1950	1
2	1950	5
3	1950	2
4	1950	2

In [137…
```
winners["decade"] = winners["year"] // 10 * 10
by_decade = (winners.groupby("decade")["start_position"]
                    .apply(lambda s: (s == 1).mean() * 100).round(1))
by_decade
```

Out[137…
```
decade
1950    40.2
1960    37.0
1970    36.8
1980    28.2
1990    42.6
2000    50.0
2010    50.5
2020    52.3
Name: start_position, dtype: float64
```

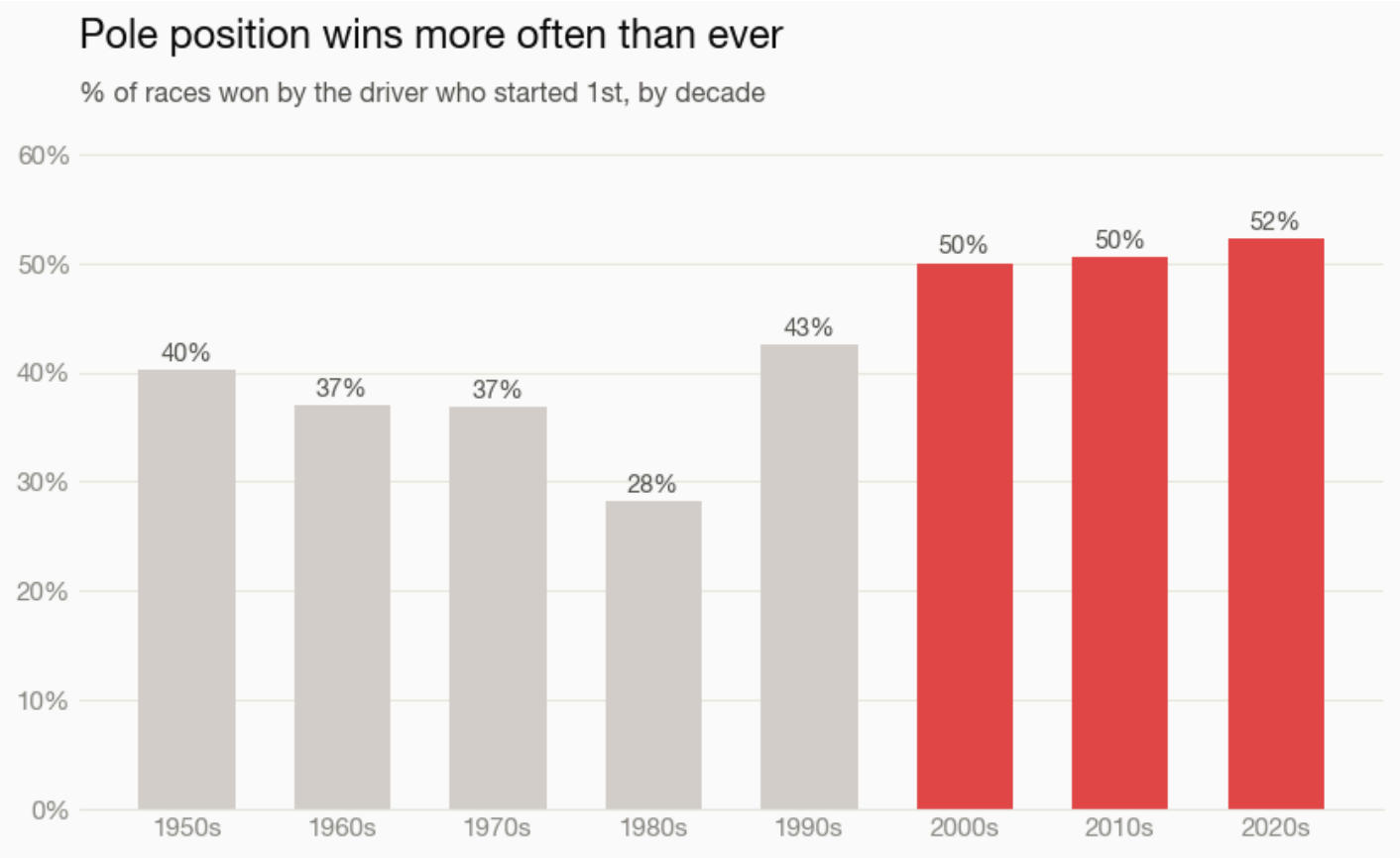
In [139…
```
fig, ax = new_chart("Pole position wins more often than ever",
                    "% of races won by the driver who started 1st, by decade")

bars = ax.bar([f"{d}s" for d in by_decade.index], by_decade.values, width=0.62,
              color=[RED if d >= 2000 else NEUTRAL for d in by_decade.index])
```

```
label_bars(ax, bars, "{:.0f}%")

ax.set_ylim(0, by_decade.max() * 1.18)
ax.yaxis.set_major_formatter(mtick.PercentFormatter(decimals=0))

save(fig, "1_pole_by_decade")
plt.show()
```



> **Takeaway:** In the 1950s–70s the pole sitter won 37–40% of races, dropping to a low of **28%** in the 1980s. Since 2000 it's been **50% or more**, reaching **52%** in the 2020s. Starting first matters more than ever.

In [175…

```
gaps = sql("""
SELECT
    CASE WHEN r1.constructorId = r2.constructorId
         THEN 'Teammates (same car)' ELSE 'Different teams' END   AS pair_type,
    ROUND(AVG(ABS(r1.positionOrder - r2.positionOrder)), 1)        AS avg_gap
FROM results r1
JOIN results r2 ON r1.raceId = r2.raceId AND r1.driverId < r2.driverId
JOIN races ra   ON ra.raceId = r1.raceId
WHERE r1.position IS NOT NULL AND r2.position IS NOT NULL
  AND ra.year >= 2000
GROUP BY pair_type
ORDER BY avg_gap
""")

gaps
```

Out [175…

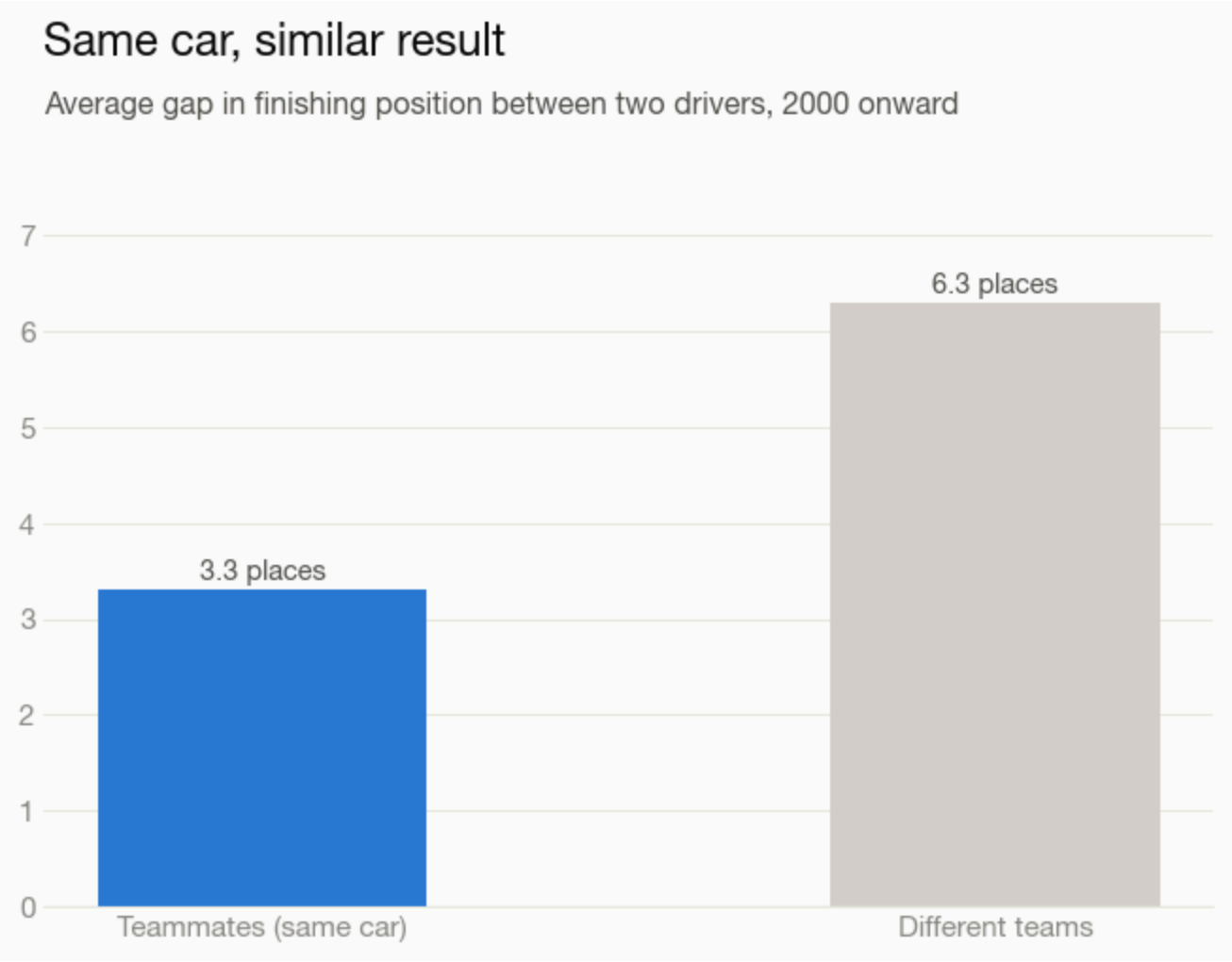
	pair_type	avg_gap
0	Teammates (same car)	3.3
1	Different teams	6.3

In [143…

```
fig, ax = new_chart(
    "Same car, similar result",
    "Average gap in finishing position between two drivers, 2000 onward",
    figsize=(8, 5.2))

bars = ax.bar(gaps["pair_type"], gaps["avg_gap"],
              color=[BLUE, NEUTRAL], width=0.45)
label_bars(ax, bars, "{:.1f} places")
ax.set_ylim(0, gaps["avg_gap"].max() * 1.25)

save(fig, "2_teammates_vs_rivals")
plt.show()
```



> **Takeaway:** Teammates in identical cars finish **3.3 places** apart vs. **6.3** for drivers on different teams. Sharing a car cuts the gap roughly in half, so the car explains most of the result, while the remaining gap reflects driver skill.

3. What else wins races?

The pit crew, and the team's overall dominance in each era.

In [146…

```
pits = sql("""
SELECT
    r.raceId,
    r.positionOrder              AS finish,
    AVG(p.milliseconds) / 1000.0 AS avg_pit_sec
FROM pit_stops p
JOIN results r ON r.raceId = p.raceId AND r.driverId = p.driverId
WHERE p.milliseconds < 60000 AND r.position IS NOT NULL
GROUP BY r.raceId, r.driverId, r.positionOrder
""")

pits.head()
```

Out[146…

	raceId	finish	avg_pit_sec
0	841	2	23.213000
1	841	12	24.046000
2	841	4	24.055000
3	841	7	24.145667
4	841	13	24.873500

In [164…

```
race_avg = pits.groupby("raceId")["avg_pit_sec"].transform("mean")
pits["vs_race_avg"] = pits["avg_pit_sec"] - race_avg

rho, p_value = spearmanr(pits["vs_race_avg"], pits["finish"])
print(f"Spearman correlation: {rho:.2f}  (p-value: {p_value:.2g})")

labels = ["Fastest 25%", "Faster", "Slower", "Slowest 25%"]
pits["group"] = pd.qcut(pits["vs_race_avg"], 4, labels=labels)
pit_summary = (pits.groupby("group", observed=True)["finish"]
                   .mean().round(1))
pit_summary
```

```
Spearman correlation: 0.40  (p-value: 3.9e-190)
```

Out[164…

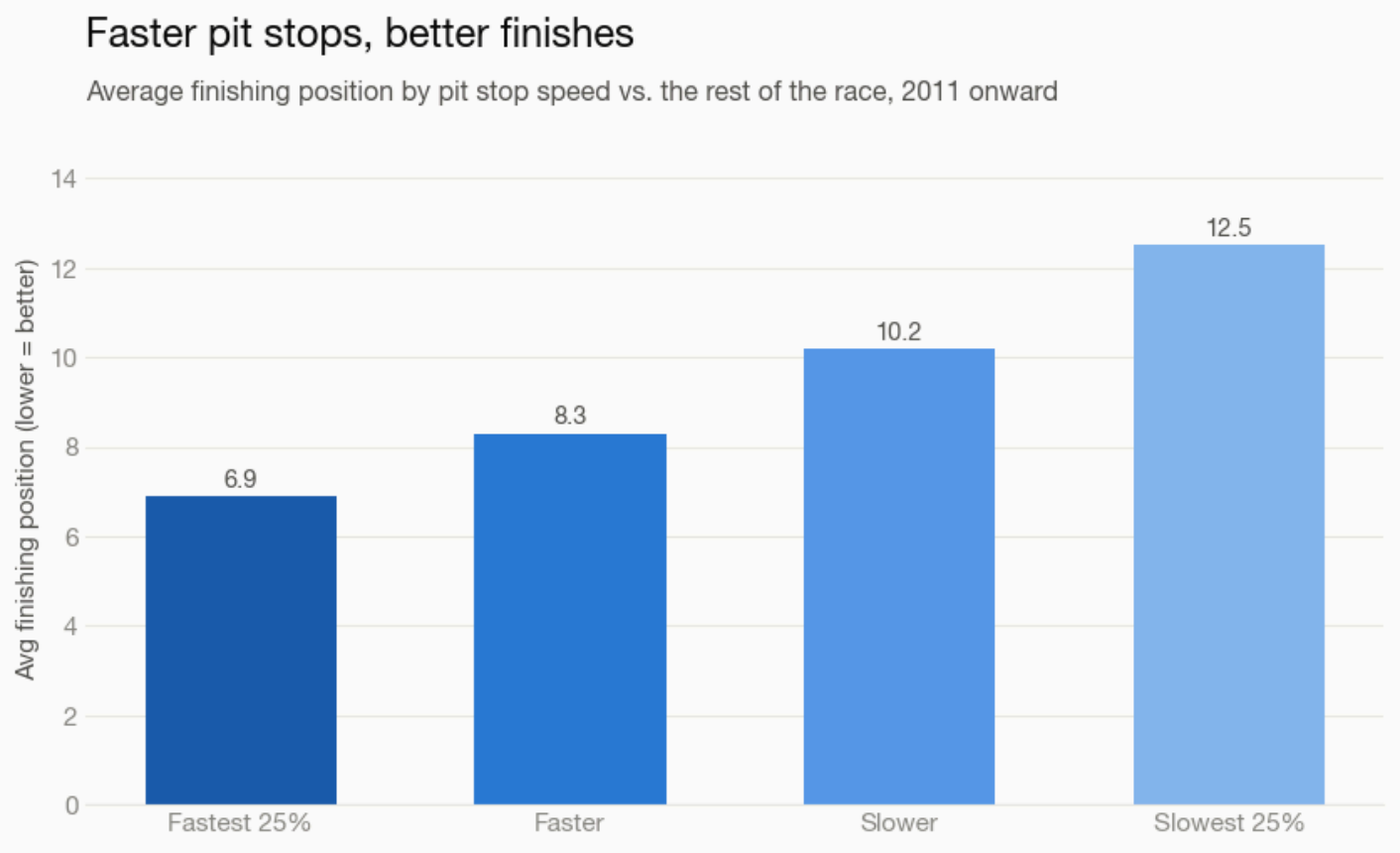
```
group
Fastest 25%     6.9
Faster          8.3
Slower         10.2
Slowest 25%    12.5
Name: finish, dtype: float64
```

In [166…

```
fig, ax = new_chart(
    "Faster pit stops, better finishes",
    "Average finishing position by pit stop speed vs. the rest of the race, "
    "2011 onward")

bars = ax.bar(pit_summary.index.astype(str), pit_summary.values,
              color=BLUE_RAMP, width=0.58)
label_bars(ax, bars, "{:.1f}")
ax.set_ylim(0, pit_summary.max() * 1.2)
ax.set_ylabel("Avg finishing position (lower = better)")

save(fig, "3_pit_stops")
plt.show()
```



> **Takeaway:** Drivers with the fastest 25% of pit stops finish **6.9th** on average vs. **12.5th** for the slowest, about **5.6 places** better. The link is moderate and highly statistically significant (Spearman correlation = 0.40, p < 0.001).
>
> **Caveat:** correlation, not causation. Top teams tend to have both faster pit crews and faster cars.

In [168…

```
dominant = sql("""
WITH team_wins AS (
    SELECT (ra.year / 10) * 10 AS decade, c.name AS team, COUNT(*) AS wins
    FROM results r
    JOIN races ra        ON ra.raceId = r.raceId
    JOIN constructors c  ON c.constructorId = r.constructorId
    WHERE r.positionOrder = 1
    GROUP BY decade, team
),
ranked AS (
    SELECT *,
           SUM(wins) OVER (PARTITION BY decade) AS races_in_decade,
           RANK() OVER (PARTITION BY decade
                        ORDER BY wins DESC)     AS rnk
    FROM team_wins
)
SELECT decade, team,
       ROUND(100.0 * wins / races_in_decade, 1) AS share
FROM ranked
WHERE rnk = 1
ORDER BY decade
""").drop_duplicates("decade")

dominant
```

Out[168…

	decade	team	share
0	1950	Ferrari	34.5
1	1960	Lotus-Climax	22.0
2	1970	Ferrari	25.7
3	1980	McLaren	35.9
4	1990	Williams	37.7
5	2000	Ferrari	48.9
6	2010	Mercedes	47.0
7	2020	Red Bull	56.1

In [170…

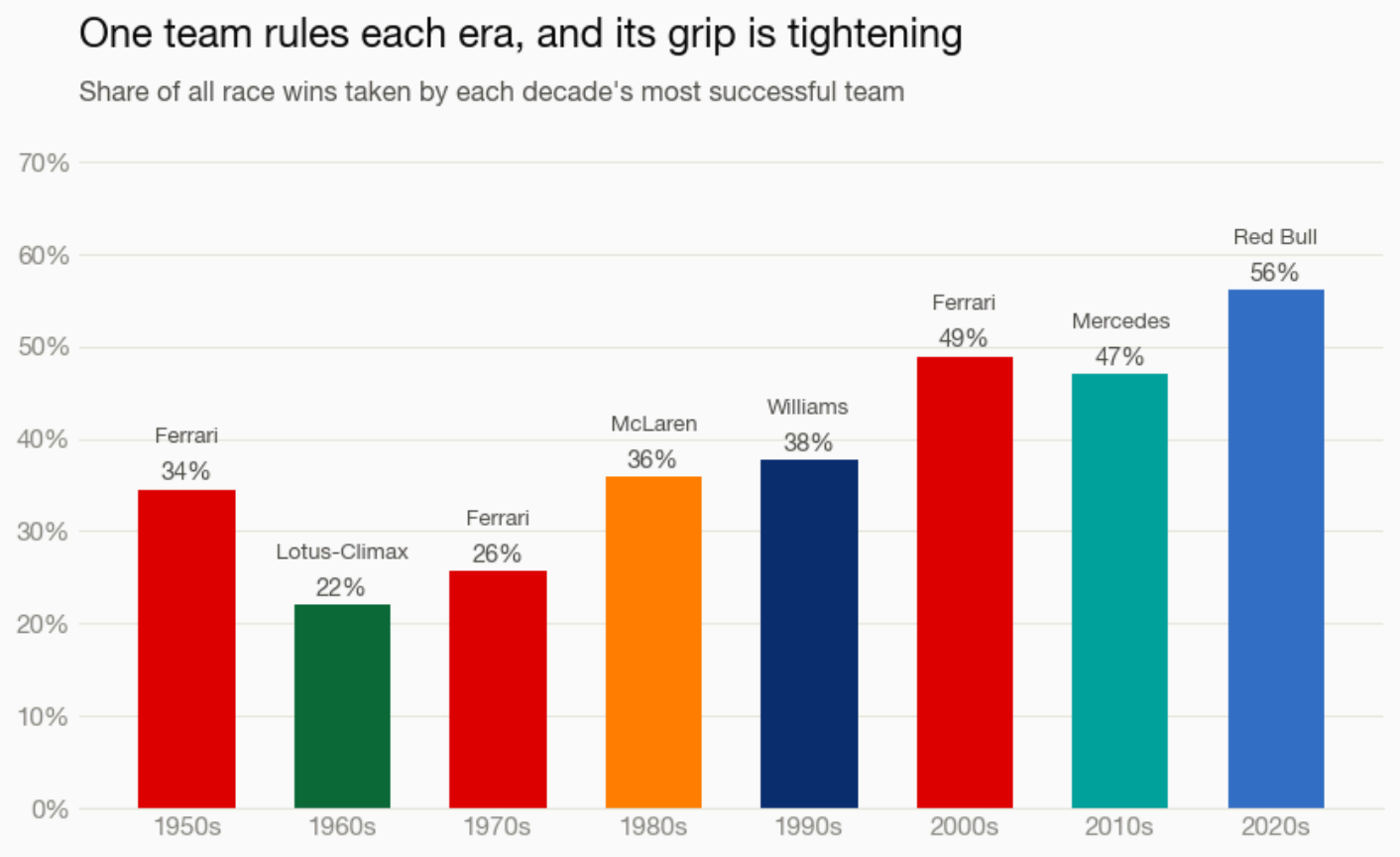
```
fig, ax = new_chart(
    "One team rules each era, and its grip is tightening",
    "Share of all race wins taken by each decade's most successful team")

decades = [f"{d}s" for d in dominant["decade"]]
colors = [team_color(t) for t in dominant["team"]]
bars = ax.bar(decades, dominant["share"], width=0.62, color=colors)

for bar, team, share in zip(bars, dominant["team"], dominant["share"]):
    x = bar.get_x() + bar.get_width() / 2
    ax.annotate(f"{share:.0f}%", (x, share), xytext=(0, 4),
                textcoords="offset points", ha="center",
                fontsize=10.5, fontweight="bold", color=INK_2)
    ax.annotate(team, (x, share), xytext=(0, 20),
                textcoords="offset points", ha="center",
                fontsize=9.5, color=INK_2)

ax.set_ylim(0, dominant["share"].max() * 1.3)
ax.yaxis.set_major_formatter(mtick.PercentFormatter(decimals=0))

save(fig, "4_team_dominance")
plt.show()
```



> **Takeaway:** The most successful team's share of wins has grown from about **22–26%** in the 1960s–70s to **roughly half** since 2000: Ferrari (49%) in the 2000s, Mercedes (47%) in the 2010s, and Red Bull (**56%**) so far in the 2020s. *(The 2020s only include seasons through 2024.)*

4. Conclusion

What wins Formula 1 races? In the modern era, mostly the car. The fastest car qualifies at the front, and with modern reliability it rarely breaks down, so pole converts to a win about half the time. Teammates in the same car finish close together, top teams pair fast cars with fast pit crews, and a single team now takes about half of all wins. Driver skill still makes a measurable difference within the same car.

From 14 CSV files to these insights: Python loaded the raw files and fixed missing values; a SQLite database brought them together; SQL (joins, a self-join, CTEs, and window functions) pulled each answer; Python handled the statistics and charts.

Limitations: pit stop data starts in 2011; the driver-vs-car test only uses races where both drivers finished; findings are correlations, not proof of cause.